PayWallet

Hệ thống Ví điện tử & Thanh toán — Tài liệu hướng dẫn Demo phỏng vấn

Vị trí ứng tuyển: **Backend Developer (Java/Spring)** — tham gia với tư cách Fresher. Tài liệu này mô tả dự án đã xây dựng, ánh xạ với yêu cầu công việc (JD), hướng dẫn cài đặt & chạy, và **kịch bản demo trực tiếp** kèm các điểm kỹ thuật để trả lời phỏng vấn.

LƯU Ý	Đây là dự án **mô phỏng** cho mục đích học tập/demo. Dùng **tiền ảo**, không kết nối cổng thanh toán thật, không xử lý thẻ/tiền thật.

DEMO TRỰC TUYẾN	
Web demo	**https://sonddd.site**
Swagger API	https://sonddd.site/swagger-ui.html
Mã nguồn	https://github.com/sonddd17/payment-wallet-demo
Tài khoản	alice / alice123 (USER) · admin / admin123 (ADMIN)

1. Tổng quan dự án

PayWallet là một ứng dụng **backend REST API** mô phỏng ví điện tử. Người dùng có thể đăng ký, đăng nhập, nạp tiền, chuyển tiền cho người khác, thanh toán cho merchant, xem lịch sử giao dịch; quản trị viên xem toàn bộ giao dịch và báo cáo tổng hợp. Kèm theo một **trang web demo** để thao tác trực quan.

Công nghệ sử dụng:

- **Java 21, Spring Boot 3.4** — nền tảng backend.
- **Spring MVC** — xây dựng REST API; **Spring Data JPA + Hibernate** — ORM, thao tác CSDL.
- **Spring Security + JWT** — xác thực/phân quyền (USER/ADMIN), mật khẩu băm BCrypt.
- **H2** (mặc định, chạy nhanh) và **PostgreSQL** (profile Docker) — hệ quản trị CSDL.
- **JUnit 5 + Mockito** (unit test) và **Spring Boot Test + MockMvc** (integration test).
- **Maven** (build), **Docker/Docker Compose** (đóng gói & triển khai), **Jenkins** (CI/CD), **Git**.
- **Swagger/OpenAPI** — tài liệu API tương tác để demo.

2. Dự án đáp ứng JD như thế nào

Bảng dưới ánh xạ từng yêu cầu trong JD với phần tương ứng trong dự án — dùng để trả lời khi được hỏi “em đã làm gì liên quan đến…”.

Yêu cầu trong JD	Thể hiện trong dự án
OOP, cấu trúc dữ liệu & giải thuật	Kiến trúc phân tầng Controller–Service–Repository, tách interface/impl, DTO (record); dùng Map/List, so sánh BigDecimal, sắp xếp khóa theo id để tránh deadlock.

Yêu cầu trong JD	Thể hiện trong dự án
Thiết kế & triển khai REST API	10+ endpoint REST chuẩn (đăng ký, đăng nhập, ví, chuyển tiền, thanh toán, lịch sử, admin), response thống nhất, mã HTTP đúng ngữ nghĩa (200/201/400/401/403/404/422).
ORM (JPA, Hibernate)	4 entity (User, Wallet, Merchant, Transaction) với quan hệ 1-1, N-1; @Version (khóa lạc quan); index; Hibernate tự sinh bảng từ entity.
CSDL: truy vấn phức tạp, tối ưu	JPQL nhiều điều kiện động + phân trang (lịch sử giao dịch); truy vấn GROUP BY + SUM (báo cáo); đánh index các cột tra cứu; H2 & PostgreSQL.
Spring Boot, Core, MVC, Data JPA, Security	Toàn bộ dự án dựng trên các module này: @RestController, @Service, JpaRepository, SecurityFilterChain + JWT filter.
Maven/Ant, Docker, Jenkins	pom.xml (Maven); Dockerfile multi-stage + docker-compose (app + PostgreSQL); Jenkinsfile (checkout → test → package → build image).
Unit test & integration test (JUnit, Mockito)	PaymentServiceImplTest (Mockito, 4 ca); AuthFlowIntegrationTest (MockMvc, end-to-end). Tất cả PASS trên máy.
Git	Dự án có .gitignore chuẩn, khởi tạo Git repo, commit lịch sử rõ ràng.
SOAP API (tùy chọn trong JD)	JD ghi “REST hoặc SOAP”. Dự án tập trung REST (đã chạy thực tế); có thể bổ sung endpoint SOAP bằng Spring-WS nếu cần — em nắm được khái niệm WSDL/XSD.

3. Cài đặt môi trường & chạy

Máy demo đã có **JDK (Java)** và **Git**.

Cách nhanh nhất — chạy file jar đã build sẵn (KHÔNG cần cài Maven):

```
cd D:\JavaFS\payment-wallet-demo
java -jar target\payment-wallet-demo.jar
```

Nếu muốn build lại từ mã nguồn thì cài thêm Maven theo các bước dưới.

Bước 1 — Cài Maven (mở PowerShell):

```
winget install Apache.Maven
# M■ l■i c■a s■ terminal sau khi cài, ki■m tra:
mvn -v
```

Bước 2 — Chạy ứng dụng (tại thư mục dự án D:\JavaFS\payment-wallet-demo):

```
mvn spring-boot:run
```

Ứng dụng chạy ở cổng 8080. Mở trình duyệt:

- Trang web demo: **http://localhost:8080**
- Tài liệu API (Swagger): **http://localhost:8080/swagger-ui.html**
- Xem CSDL (H2 Console): **http://localhost:8080/h2-console** — JDBC URL: `jdbc:h2:mem:paywallet`, user `sa`, không mật khẩu.

Cách khác — chạy bằng Docker (giống production, kèm PostgreSQL), nếu đã cài Docker Desktop:

```
docker compose up --build
```

Tài khoản tạo sẵn để demo:

Tài khoản	Mật khẩu	Vai trò	Số dư ban đầu
admin	admin123	ADMIN	0
alice	alice123	USER	750.000
bob	bob123	USER	500.000

Merchant demo: **COFFEE** (Highlands Coffee), **SHOP** (Demo Online Shop).

Bản đang chạy thật trên Internet (production):

Dự án đã được triển khai tại **https://sonddd.site**. Nếu không tiện chạy trên máy cá nhân, có thể demo trực tiếp trên link này.

- VPS AlmaLinux 8, 1 CPU, 1GB RAM (thêm 2GB swap).
- Docker Compose chạy 2 container: ứng dụng Spring Boot (JRE 21, giới hạn heap 384MB) và PostgreSQL 16 (dữ liệu lưu trong volume).
- Ứng dụng chỉ mở cổng cho localhost; **Nginx** làm reverse proxy ra Internet.
- **Cloudflare** cung cấp HTTPS và tự chuyển HTTP sang HTTPS.
- Mật khẩu CSDL và JWT secret sinh ngẫu nhiên, lưu trong file .env trên server, không đưa lên GitHub.

4. Kịch bản demo trực tiếp (gợi ý thao tác & lời nói)

Thực hiện lần lượt trên trang **http://localhost:8080**. Mỗi bước có thao tác và câu gợi ý để trình bày.

Bước 1: Đăng nhập

Thao tác: Trang đã điền sẵn alice / alice123 → bấm “Đăng nhập”. Nhật ký bên dưới hiện JWT trả về.

Nên nói: *“Hệ thống xác thực bằng JWT. Sau khi đăng nhập, server trả về access token; các request sau gửi kèm token trong header Authorization.”*

Bước 2: Xem số dư ví

Thao tác: Sau đăng nhập tự hiện số dư 750.000 VND và trạng thái ví ACTIVE.

Nên nói: *“Mỗi người dùng có đúng một ví (quan hệ 1-1). Số dư lưu bằng BigDecimal để tránh sai số khi tính tiền.”*

Bước 3: Nạp tiền

Thao tác: Nhập 100000 ở ô “Nạp tiền” → bấm “Nạp vào ví”. Số dư tăng, lịch sử xuất hiện giao dịch TOPUP.

Nên nói: *“Thao tác nạp được bọc trong @Transactional và khóa ví (pessimistic lock) khi cập nhật số dư để an toàn khi có truy cập đồng thời.”*

Bước 4: Chuyển tiền

Thao tác: Người nhận: bob, số tiền 50000 → bấm "Chuyển". Số dư alice giảm, giao dịch TRANSFER hiện ra.

Nên nói: *"Chuyển tiền là nghiệp vụ nhạy cảm: em kiểm tra số dư, khóa cả hai ví theo thứ tự id tăng dần để tránh deadlock, và cập nhật nguyên tử trong một transaction."*

Bước 5: Thanh toán merchant

Thao tác: Chọn merchant COFFEE, nhập 35000 → bấm "Thanh toán". Giao dịch PAYMENT xuất hiện.

Nên nói: *"Thanh toán trừ tiền ví người dùng và cộng vào tài khoản merchant trong cùng một giao dịch. Mỗi giao dịch có mã tham chiếu duy nhất và hỗ trợ idempotency-key chống trừ tiền 2 lần."*

Bước 6: Xem lịch sử (phân trang & lọc)

Thao tác: Bấm "Tải lịch sử" để xem bảng giao dịch gần nhất (đã phân trang).

Nên nói: *"Lịch sử dùng truy vấn JPQL nhiều điều kiện động (loại, trạng thái, khoảng thời gian) kết hợp phân trang của Spring Data."*

Bước 7: Phân quyền & báo cáo ADMIN

Thao tác: Đăng xuất, đăng nhập admin / admin123 → ô "Báo cáo (ADMIN)" hiện ra → bấm "Xem báo cáo".

Nên nói: *"Chỉ ADMIN truy cập được /api/admin/** (phân quyền theo vai trò). Báo cáo dùng truy vấn GROUP BY + SUM để tổng hợp theo loại giao dịch. Nếu USER gọi API này sẽ bị trả 403."*

Bước 8: Tài liệu API với Swagger

Thao tác: Mở http://localhost:8080/swagger-ui.html, bấm "Authorize" dán token, thử gọi vài endpoint.

Nên nói: *"Em tích hợp OpenAPI/Swagger để tài liệu hóa và thử API ngay trên trình duyệt — thuận tiện cho tester và frontend."*

5. Điểm kỹ thuật nổi bật (để trả lời phỏng vấn)

1) Toàn vẹn dữ liệu khi giao dịch tiền

Mọi thao tác đổi số dư nằm trong `@Transactional`. Khi chuyển/thanh toán, ví được lấy kèm khóa ghi (`PESSIMISTIC_WRITE`) và khóa theo thứ tự id tăng dần để tránh deadlock. Entity còn có `@Version` (khóa lạc quan) làm lớp phòng thủ thứ hai.

2) Idempotency — chống trừ tiền trùng

Mỗi yêu cầu có thể gửi kèm *idempotency-key*. Nếu key đã tồn tại, hệ thống trả lại giao dịch cũ thay vì thực hiện lần nữa — tránh việc người dùng bấm 2 lần hoặc mạng timeout gây trừ tiền 2 lần.

3) Bảo mật

Xác thực bằng JWT (stateless), mật khẩu băm BCrypt, phân quyền USER/ADMIN theo URL và bằng `@PreAuthorize`. Có bộ lọc JWT đọc token, xác thực và nạp quyền vào SecurityContext.

4) Xử lý lỗi tập trung

`@RestControllerAdvice` chuyển mọi ngoại lệ thành cấu trúc lỗi thống nhất với mã HTTP phù hợp (404 không tìm thấy, 409 trùng, 422 thiếu số dư, 400 sai dữ liệu + chi tiết từng trường).

5) Kiểm thử

Unit test dùng Mockito giả lập repository để kiểm thử nghiệp vụ mà không cần CSDL (chuyển tiền thành công, thiếu số dư, tự chuyển cho mình, idempotency). Integration test khởi động toàn bộ Spring Boot + H2 và gọi API thật qua MockMvc.

6) Triển khai thực tế

Ứng dụng chạy trên VPS bằng Docker Compose sau Nginx và Cloudflare. Vì VPS chỉ có 1GB RAM, em build jar trên máy rồi đưa lên server, image chỉ chứa JRE và giới hạn heap. Cloudflare đang ở chế độ SSL Flexible; hướng nâng cấp là cài chứng chỉ Let's Encrypt lên server rồi chuyển sang Full (strict) để mã hóa cả đoạn Cloudflare tới server.

6. Danh sách endpoint chính

Method	Đường dẫn	Chức năng	Quyền
POST	/api/auth/register	Đăng ký + tạo ví	Công khai
POST	/api/auth/login	Đăng nhập, nhận JWT	Công khai
GET	/api/wallet/me	Xem số dư	USER
POST	/api/wallet/topup	Nạp tiền	USER
POST	/api/payments/transfer	Chuyển tiền	USER
POST	/api/payments/pay	Thanh toán merchant	USER
GET	/api/payments/history	Lịch sử (lọc + phân trang)	USER
GET	/api/merchants	Danh sách merchant	Công khai
GET	/api/admin/transactions	Toàn bộ giao dịch	ADMIN
GET	/api/admin/report	Báo cáo tổng hợp	ADMIN

Chúc bạn demo thành công! — Tài liệu cho sonddd, 2026.